MASTER PARISIEN DE RECHERCHE OPÉRATIONNELLE

Rapport de projet

PORT

Par

BACCAR Bedis
ESTEVE Erwann
VERBOIS Maël

Le code utilisé dans ce projet est entièrement disponible à l'adresse suivante : `https://github.com/maelVerbois0/PORT_BNP.git`*. Les instructions d'utilisation sont détaillées dans le fichier README.md.*

1 Introduction

Notre implémentation s'appuie sur le code fourni, auquel nous avons apporté les ajouts et modifications suivants :

— refonte en profondeur de l'architecture logicielle via l'abstraction de la logique et la séparation des responsabilités au sein de classes dédiées ;
— séparation plus claire entre les composants de modélisation et de résolution, ainsi que la création de composants spécifiques à la logique de *Branch-and-Price* ;
— implémentation et comparaison de plusieurs stratégies de branchement pour le *Branch-and-Price*.

L'objectif de ce rapport est de présenter la démarche de conception, les choix algorithmiques, les décisions d'implémentation, ainsi que leurs impacts sur le comportement du solveur.

2 Refactorisation du code

Nous avons entrepris une refonte du code préexistant afin de faciliter le développement des nouvelles méthodes de *Branch-and-Bound* à ajouter. Ce choix a notamment été motivé par la confusion, dans le code initial, entre la modélisation de l'instance et la résolution du problème.

La présence de tous les fichiers dans un unique dossier masquait la responsabilité de chaque partie du code, et l'absence de classes rendait son utilisation difficile.

Nous avons donc jugé préférable de repartir sur une base plus saine. Certaines classes existantes ont pu être conservées telles quelles, mais la plupart ont été modifiées ou créées de toutes pièces.

À toutes fins utiles, nous proposons dans la suite de cette section une vue d'ensemble de la nouvelle architecture, dont l'idée générale s'articule ainsi :

— une classe **Parser** : chargée de lire (parser) les fichiers d'instances ;
— une classe **TimeSpaceGraph** : permettant de représenter le graphe spatio-temporel sur lequel sont déterminés les itinéraires de nos trains ;
— une classe **BranchAndPriceSolver** : qui orchestre la résolution par *Branch-and-Price* (génération de colonnes, stratégie d'exploration de l'arbre, résolution du problème maître et du *pricer*).

2.1 Dossier *parser*

Le module de parsing lit les fichiers CSV d'une instance TUSP et construit une structure de données unique et exploitable par le solveur. Il s'agit de la partie du code qui a été la moins modifiée par rapport au projet original ; elle n'a fait l'objet que d'une encapsulation dans une classe afin de bien isoler sa responsabilité.

Nous avons décidé d'abandonner l'indexation originale des composants (nœuds physiques, arcs, trains) qui commençait par 1 (source de nombreuses erreurs d'accès aux tableaux), pour la remplacer par une indexation commençant par 0. Il est donc normal que les fichiers de solution utilisent des indices commençant par 0, ou que les services effectués sur le train d'indice i ne

semblent pas correspondre aux données d'entrée du train i de l'instance. À cause de ce décalage, l'indice i ne désigne plus le même train.

Rôle principal

- Lire les données brutes (services, nœuds, liens, types de trains, trains, opérations).
- Vérifier la cohérence des entrées (format, identifiants, références, bornes temporelles).
- Produire un objet `InstanceData` propre pour la suite (construction du graphe puis résolution).

Organisation

- Un utilitaire générique lit les CSV ligne par ligne.
- Chaque catégorie de données possède une fonction de parsing dédiée.
- Une fonction globale orchestre l'ordre de lecture pour respecter les dépendances entre fichiers.

2.2 Dossier *data*

Le dossier data regroupe les éléments de base du problème TUSP. Il sert de couche de modélisation des données d'entrée avant la construction du graphe et la résolution par *Branch-and-Price*.

Fichiers présents

- `InstanceData.h`
- `PNode.h`, `PNode.cpp`
- `PLink.h`, `PLink.cpp`
- `Service.h`, `Service.cpp`
- `Train.h`, `Train.cpp`

Contenu par composant

- `InstanceData.h` : définit une structure agrégée `InstanceData` contenant les informations principales de l'instance (nœuds, types de nœuds, liens, trains, types de trains, services, affectations).
- `PNode.h/.cpp` :
  - classe `PNode` (identifiant, nom, type) : représente un nœud physique du réseau ;
  - classe `NodeType` (propriétés du type de nœud : IO, parking, dwelling, services autorisés, liste des nœuds de ce type).
- `PLink.h/.cpp` : classe `PLink` représentant un lien physique du réseau (id, origine, destination, longueur).
- `Service.h/.cpp` : structure `Service` pour les opérations de maintenance/service (id, nom, coût, durée).
- `Train.h/.cpp` :
  - classe `Train` (id, nom, type, horaires arrivée/départ, services requis) ;
  - classe `TrainType` (id, nom, poids de manœuvre, temps de retournement).

2.3 Dossier *graph*

Le dossier graph contient les structures du graphe espace-temps utilisé par le solveur. Il encapsule au sein d'une classe explicite la représentation du graphe modélisé dans le sujet.

Fichiers présents
- `Arc.h`, `Arc.cpp`
- `Node.h`, `Node.cpp`
- `TimeSpaceGraph.h`, `TimeSpaceGraph.cpp`

Contenu des classes
- `Arc` : représente un arc du réseau espace-temps (origine, destination, type, coût, service associé, incompatibilités).
- `Node` : représente un nœud espace-temps (nœud physique, instant, couche) et stocke les arcs entrants, sortants et incompatibles.
- `TimeSpaceGraph` : classe conteneur du graphe complet (vecteurs de nœuds/arcs, arcs par service, accessibilité par train, arcs *dummy*, données temporelles).

Rôle fonctionnel
- Encapsuler la topologie du réseau espace-temps.
- Offrir des accesseurs centralisés pour les nœuds et les arcs.
- Fournir le calcul d'identifiant de nœud virtuel via `compute_virtual_node_id`, utilisé pour indexer efficacement le graphe.

Ce dossier constitue le noyau de la modélisation : les objets de data décrivent l'instance, tandis que graph décrit l'espace de décision exploité par le *pricer*.

2.4 Dossier *builder*

Le dossier builder contient la logique de construction du graphe espace-temps à partir des données de l'instance.

Fichiers présents
- `GraphBuilder.h`
- `GraphBuilder.cpp`

Rôle de `GraphBuilder` La classe `GraphBuilder` prend un objet `InstanceData` en entrée et génère un objet `TimeSpaceGraph` prêt pour la résolution. Elle construit progressivement :
- les nœuds virtuels ;
- les différents types d'arcs ;
- les structures d'aide (arcs par service, accessibilité par train, arcs *dummy*).

Étapes principales de construction
- Calcul du pas de temps `t_s_` (PGCD des durées/temps pertinents) et de l'horizon `T_`.
- Construction des nœuds du réseau espace-temps.
- Ajout successif des arcs (arrivée/départ, *shunting*, service, *storage*, *dwelling*, *state transfer*, *dummy*).
- Retour du graphe final via la méthode `build()`.

2.5 Dossier *solver*

Ce dossier contient le cœur algorithmique du projet. Il assemble le problème maître restreint, le problème de *pricing*, la gestion de l'état des restrictions de branchement et l'exploration de l'arbre de *Branch-and-Price*.

Fichiers présents
- `Column.h`
- `BPNode.h`
- `SearchTree.h`, `SearchTree.cpp`
- `GlobalStateManager.h`, `GlobalStateManager.cpp`
- `MasterProblem.h`, `MasterProblem.cpp`
- `PricingProblem.h`, `PricingProblem.cpp`
- `BranchAndPriceSolver.h`, `BranchAndPriceSolver.cpp`

2.5.1 Rôle de chaque composant

`Column.h` : représentation des colonnes Ce fichier définit la structure `Column`, qui modélise un chemin candidat pour un train :
- identifiant de la colonne ;
- liste des arcs du graphe espace-temps ;
- services réalisés par le chemin ;
- coût total ;
- train associé.

Il contient également `ColumnPool`, un conteneur centralisant toutes les colonnes générées, doté de méthodes d'ajout, d'accès individuel et d'extraction d'un sous-ensemble de colonnes actives.

`BPNode.h` : nœud de l'arbre *Branch-and-Price* `BPNode` stocke l'état d'un nœud de recherche :
- parent/enfants ;
- profondeur ;
- borne inférieure locale ;
- statut (`UNPROCESSED`, `PROCESSING`, `FRACTIONAL`, `INTEGER`, `INFEASIBLE`, `PRUNED`) ;
- restrictions héritées de son nœud père (train(s) concerné(s), arcs interdits).

Il est important de noter que les restrictions enregistrées dans un nœud sont **incrémentales** : on ne conserve que les nouvelles restrictions imposées par le branchement depuis le nœud père, et non **l'historique complet des restrictions** depuis la racine. Cette approche permet de réduire considérablement l'empreinte mémoire de l'état courant et d'éviter les redondances.

Pour connaître l'ensemble des restrictions lors de la transition d'un nœud à un autre, il est nécessaire de calculer la trajectoire complète dans l'arbre de recherche et d'appliquer ou retirer les restrictions de manière itérative.

`SearchTree.*` : pilotage de l'exploration `SearchTree` gère :
- la création des nœuds enfants ;
- la stratégie d'exploration *Best-Bound-First* ;
- la file de priorité des nœuds ouverts ;
- les bornes globales (meilleures bornes inférieure et supérieure) ;
- la meilleure solution courante.

Ce composant fournit également un objet `Trajectory` (nœuds à annuler / nœuds à appliquer) pour assurer une transition d'état propre lors du passage d'un nœud à l'autre.

`GlobalStateManager.*` : gestion incrémentale des restrictions Ce module maintient l'état logique global induit par le branchement :
- compteur du nombre de restrictions sur les arcs ;
- index inverse arc → colonnes impactées ;
- compteur de conflits actifs par colonne.

Lorsqu'une restriction est appliquée ou retirée, il calcule la liste exacte des colonnes à désactiver/réactiver dans le maître, évitant ainsi de reconstruire entièrement le modèle à chaque nœud.

`MasterProblem.*` : relaxation maître (RMP) `MasterProblem` encapsule un modèle Gurobi et les contraintes du problème maître :
- contrainte de flot (un chemin par train) ;
- contraintes de couverture des services requis ;
- contraintes de conflits d'occupation (nœud physique, instant).

Il expose les fonctionnalités suivantes :
- résolution de la relaxation continue (LP) ;
- extraction des variables duales (flot, service, conflit) ;
- ajout dynamique de colonnes ;
- activation/désactivation de colonnes (modification des bornes supérieures des variables) ;
- tests sur la nature de la solution (entière ou fractionnaire).

`PricingProblem.*` : sous-problème de génération de colonnes `PricingProblem` calcule, train par train, le plus court chemin à coût réduit sur le graphe espace-temps, en tenant compte :
- des variables duales du maître ;
- des services requis ;
- des conflits ;
- des arcs interdits (via `GlobalStateManager`).

L'implémentation repose sur une programmation dynamique basée sur des états étendus (nœud, masque de services), avec reconstruction du chemin optimal et création d'une nouvelle colonne si le coût réduit est améliorant. Il s'agit presque exactement de l'implémentation du premier projet, adaptée à nos structures de données.

Un point notable à souligner est que la première implémentation cherchait la solution de plus faible coût réduit parmi les chemins effectuant un nombre arbitraire de services. Il était donc possible que le *pricer* propose des colonnes invalides (où tous les services requis pour un train n'étaient pas satisfaits), mais très attractives en termes de coût.

Nous avons donc corrigé ce comportement pour ne conserver que le chemin de plus faible coût satisfaisant *l'intégralité* des services requis, garantissant ainsi la validité de toutes les colonnes générées.

`BranchAndPriceSolver.*` : orchestrateur global `BranchAndPriceSolver` coordonne l'ensemble des modules :
- initialise le système (maître, *pricing*, état global, arbre, *pool*) ;
- lance la génération de colonnes à chaque nœud ;
- met à jour le statut du nœud (entier, fractionnaire, infaisable, élagué) ;

— met à jour les bornes et la meilleure solution connue (*incumbent*) ;
— crée les nœuds enfants en cas de solution fractionnaire ;
— applique ou annule les modifications d'état (*deltas*) lors de la navigation dans l'arbre.

Les méthodes de branchement sont définies dans ce composant, mais leur logique détaillée est volontairement abordée dans la section dédiée du rapport.

2.5.2 Positionnement dans l'architecture globale du projet

Dans l'architecture complète :
— le *parser* prépare les données ;
— le *builder* construit le graphe espace-temps ;
— le *solver* exploite ce graphe pour résoudre le problème par *Branch-and-Price*.

Le dossier solver constitue donc la couche de décision et d'optimisation du projet, qui s'appuie sur les couches de modélisation en amont.

3 Implémentation du *Branch-and-Price*

Comme décrit ci-dessus, l'intégralité de la logique de résolution a été implémentée dans le dossier *solver*. Il convient cependant de détailler le flux logique de la résolution et d'expliciter certains choix d'implémentation.

Dans un premier temps, voici le pseudo-code de haut niveau décrivant le déroulement global de la résolution d'une instance :

3.1 Chaîne d'exécution globale du solveur

1. Initialisation de la racine et des colonnes de base.
2. Sélection d'un nœud ouvert dans l'arbre.
3. Synchronisation de l'état (restrictions actives) avec ce nœud.
4. Si les conditions sont remplies, lancement d'heuristiques primales pour améliorer la meilleure solution connue.
5. Boucle de génération de colonnes : résolution maître → extraction des variables duales → *pricing* → ajout de colonnes.
6. Arrêt de la boucle quand aucun chemin améliorant n'est trouvé ou qu'un critère d'arrêt basé sur la borne lagrangienne est atteint.
7. Qualification du nœud (entier/fractionnaire/infaisable) et mise à jour des bornes.
8. Si fractionnaire, création de deux enfants avec des restrictions complémentaires.
9. Répétition jusqu'à épuisement des nœuds ouverts.

Nous décrivons ci-après plus en détail chacune de ces étapes.

3.2 Initialisation

Initialement, le nœud racine ne possède aucune restriction et n'a pour seules colonnes que les arcs *dummy* introduits dans la modélisation pour assurer la faisabilité initiale.

3.3 Sélection des nœuds

Nous avons décidé d'opter pour une stratégie d'exploration de l'arbre dite **Best Bound First** (BBF).

Au cours de l'exploration, nous maintenons une file de priorité des nœuds actifs triée par ordre croissant. Ainsi, nous explorons systématiquement le nœud ayant la plus petite borne inférieure connue.

Il y a deux raisons principales à ce choix. Dans un premier temps, on peut espérer que ce nœud nous donne, si l'on trouve une solution entière parmi ses fils, une solution de faible valeur. Deuxièmement, il est facile de remarquer que dans un *Branch-and-Bound*, la borne inférieure de la relaxation d'un fils est toujours supérieure ou égale à celle de son père. Cette logique favorise donc une exploration équilibrée de l'arbre.

C'est un point particulièrement favorable pour notre logique d'application itérative des modifications d'état. En effet, un arbre équilibré est globalement de moindre profondeur ; en moyenne, le passage d'un nœud à un autre nécessite donc d'appliquer moins de modifications incrémentales.

3.4 Synchronisation de l'état

Une fois qu'un nœud a été sélectionné pour être traité, il faut synchroniser l'état de l'ensemble des structures de données pour que toutes les restrictions actives à ce nœud soient bien prises en compte. Comme mentionné précédemment, les restrictions sont stockées incrémentalement dans les nœuds. Pour passer d'un nœud à l'autre, on calcule la trajectoire dans l'arbre (c'est-à-dire que l'on trouve le **Lowest Common Ancestor** ou LCA) : les restrictions du chemin montant sont annulées, et celles du chemin descendant sont appliquées via une mise à jour incrémentale des conflits dans le **GlobalStateManager** et des bornes des colonnes dans le **MasterProblem**.

3.5 Heuristiques primales

Deux heuristiques primales basées sur le *Branch-and-Price* ont été implémentées afin de trouver des solutions réalisables de *pricing*. Elles sont déclenchées tous les 5 nœuds.

Plongée (*diving*) La variable fractionnaire la plus proche de 1 est fixée en interdisant au train correspondant tous les arcs hors de la colonne choisie. Une génération de colonnes rapide est relancée, et le processus est répété jusqu'à intégrité ou infaisabilité. Les fixations sont ensuite annulées par dépilage.

Heuristique MIP globale Toutes les colonnes du *pool* sont réactivées, les variables converties en binaires, et Gurobi est lancé avec une limite de temps de 15 secondes. Si une solution améliorante est trouvée, la meilleure solution connue (*incumbent*) est mise à jour. Le modèle est ensuite reconverti en continu et les colonnes interdites sont de nouveau désactivées.

3.6 Génération de colonnes

Il s'agit d'une boucle de génération de colonnes classique. À chaque nœud, la boucle résout le problème maître restreint (RMP), extrait les variables duales, résout le *pricing* pour tous les trains et ajoute les colonnes améliorantes.

Il est plus intéressant de se pencher sur les conditions d'arrêt de cette génération de colonnes.

La première, et la plus évidente, est qu'il n'existe plus de chemin de coût réduit négatif. Cependant, cette condition est assez coûteuse à vérifier en pratique. On observe souvent un effet de traîne (**tailing off**) : le solveur passe beaucoup de temps à clôturer la génération de colonnes alors qu'il possède déjà une très bonne estimation de la borne inférieure.

Pour formaliser cette idée, on introduit la notion de borne et de gap lagrangiens. On appelle borne lagrangienne L la borne inférieure suivante, obtenue à chaque itération de notre *pricing* :

$$L = Z_{RMP} + \sum_{k \in \mathcal{K}} \bar{c}_k^*$$

avec Z_{RMP} la valeur du problème maître restreint à l'itération courante et $\bar{c}_k^*$ le coût réduit minimum du train k.

Cette quantité constitue une borne inférieure valide de notre relaxation continue. Ainsi, si la valeur de cette borne est supérieure à notre meilleure solution courante, on peut arrêter la génération de colonnes et élaguer (couper) le nœud.

S'il s'avère que le gap lagrangien :

$$G_{\text{Lagr.}} = \frac{Z_{RMP} - L}{L}$$

est inférieur à un certain seuil, on est certain d'avoir une bonne approximation de notre relaxation. Le phénomène de *tailing off* apparaît lorsque ce gap est très faible, mais que la génération de colonnes doit encore ajouter de nombreux chemins à coût réduit négatif de faible valeur avant de pouvoir le fermer complètement. Afin de lutter contre cet ajout excessif de colonnes tout en conservant les plus intéressantes, nous avons mis en place un seuil adaptatif d'arrêt utilisant ce gap, qui dépend de la profondeur de l'arbre pour accélérer l'exploration.

Profondeur	**Gap de terminaison**
0 (racine)	0,0 % (résolution exacte)
1 – 5	0,1 %
6 – 10	0,5 %
> 10	1,0 %

TABLE 1 – Seuil adaptatif d'arrêt de la génération de colonnes.

3.7 Stratégies de branchement

La méthode `choose_branching_variable` tente d'abord la règle primaire *Service Split Branching* ; en cas d'échec, la règle de secours *Quantum Train Branching* est activée via une exception.

3.7.1 Règle primaire : Service Split Branching

Pour chaque train fractionnaire, on calcule le flux agrégé sur les arcs de chaque service, indexé par instant de démarrage. On cherche un instant de coupure t^* minimisant le score :

$$\text{score}(t_i) = \frac{|F_{\leq t_i} - F_{\text{total}}/2|}{t_{i+1} - t_i}$$

où $F_{\leq t_i}$ est le flux cumulé jusqu'à t_i. Ce critère favorise simultanément l'équilibre des deux branches (numérateur faible) et une coupure temporellement bien séparée (dénominateur grand). La branche gauche interdit les arcs du service démarrant après t^* ; la branche droite interdit ceux démarrant au plus tard à t^*. Si, pour un train donné, l'ensemble des services n'admet qu'un unique temps de réalisation, une exception est levée et le solveur bascule sur la règle de secours.

3.7.2 Règle de secours : Quantum Train Branching

Pour chaque train fractionnaire, on identifie le premier instant où les chemins fractionnaires divergent. Les arcs sortants à ce point sont triés par flux décroissant et répartis en deux partitions par un algorithme glouton de type LPT (affectation au groupe le plus léger). Le train dont le déséquilibre $|S_A - S_B|$ est minimal est retenu, et les partitions définissent les interdictions respectives des deux branches.

4 Résultats expérimentaux

Cette section présente les premiers résultats obtenus sur quatre instances. La limite de temps fixée est de 3 600 secondes.

4.1 Instances

Dans un premier temps, nous nous sommes concentrés sur la première instance de chaque groupe. Cette approche nous a permis d'évaluer l'efficacité et le comportement de notre solveur sur des instances présentant un nombre relativement restreint de trains et d'affectations, à savoir :

- `small_station-scenario_1` (8 trains, 14 175 arcs) ;
- `medium_station-scenario_1` (15 trains, 35 801 arcs) ;
- `Variant_station-scenario_1` (10 trains, 13 296 arcs) ;
- `large_station-scenario_1` (25 trains, 76 390 arcs).

4.2 Résultats

Table 2 – Résultats du *Branch-and-Price* sur les instances de calibration.

Instance	Trains	Arcs	Root LB	LB	UB	Gap (%)	Temps (s)	TTFS (s)	Nœuds	Master (s)	Pricing (s)	Heur. (s)
small_1	8	14 175	1959,5	1965,0	1965,0	0,00	0,46	0,11	24	0,21	0,10	0,21
medium_1	15	35 801	3974,0	3974,0	3974,0	0,00	1,08	0,46	13	0,49	0,37	0,57
large_1	25	76 390	7264,0	7264,0	7264,0	0,00	32,96	2,70	23	4,19	3,89	29,36
Variant_1	10	13 296	2370,0	2383,5	2437,0	2,24	3607,49	0,38	1 736	94,73	12,73	3546,7

4.3 Analyse

Instances résolues à l'optimalité Les instances `small`, `medium` et `large` sont résolues à l'optimalité prouvée avec des arbres de moins de 25 nœuds. Sur `medium`, la borne racine coïncide avec l'optimum (3 974) : la relaxation continue est déjà entière et les 13 nœuds ne servent qu'à le confirmer. Sur `small`, un léger écart à la racine (1 959,5 contre 1 965) génère l'exploration de 24 nœuds. `large`, malgré ses 25 trains et 76 390 arcs, est résolue en 33 secondes avec 23 nœuds, la borne racine étant également exacte (7 264).

Sur cette dernière instance, les heuristiques représentent 89 % du temps de calcul (29,4 s). Le temps jusqu'à la première solution entière (TTFS) est de 2,7 secondes — plus élevé que sur les instances plus petites, ce qui reflète l'augmentation de la complexité du problème liée à la taille de l'instance. On remarque sur ces instances résolues à l'optimalité que la borne inférieure obtenue à la racine est d'excellente qualité, voire exacte. Cela justifie le choix de consacrer une part importante du temps de calcul à trouver une solution heuristique primale de bonne qualité, plutôt que d'essayer d'améliorer la borne.

Instance `Variant_station` Cette instance résiste à la résolution exacte malgré sa taille modeste (10 trains, 13 296 arcs, soit moins d'arcs que `small`). Après 3 607 secondes et 1 736 nœuds explorés, le gap résiduel est de 2,24 %. La décomposition du temps est frappante : les heuristiques accaparent 98,3 % du temps total (3 546,7 s), contre 2,6 % pour le maître et 0,35 % pour le *pricing*.

L'heuristique MIP globale, relancée toutes les 5 itérations sur un *pool* de colonnes croissant, constitue un véritable goulot d'étranglement. Un déclenchement conditionné à la stagnation de la borne supérieure, plutôt qu'à une fréquence fixe, ne réduirait sans doute pas ce coût ; en effet, on observe que malgré notre insistance à trouver une solution primale de bonne qualité, il s'avère très difficile de la faire baisser.

Bilan Ces résultats montrent que la difficulté est davantage liée à la structure combinatoire de l'instance (fractionnalité de la relaxation, nombre de conflits) qu'à sa dimension brute. Le TTFS reste inférieur à 3 secondes sur toutes les instances, confirmant l'efficacité de nos heuristiques pour trouver rapidement une solution réalisable.

4.4 Convergence de Variant_1

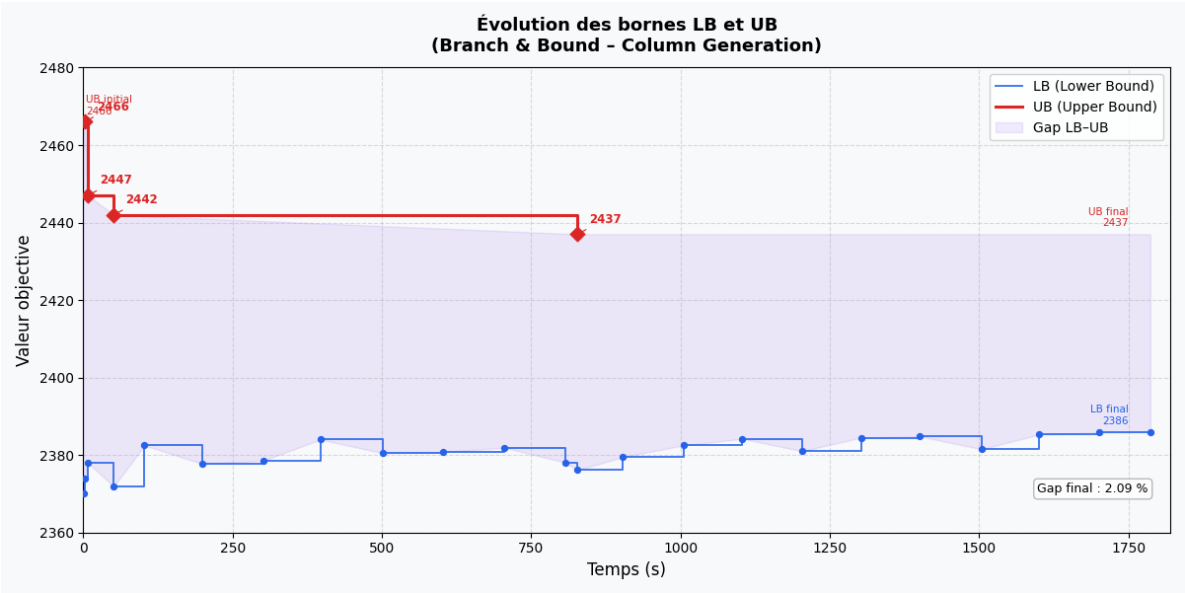

FIGURE 1 – Évolution des bornes inférieure (LB) et supérieure (UB) au cours du *Branch-and-Price* sur l'instance Variant_1.

La figure 1 illustre la dynamique de convergence sur Variant_1 sur une durée de 1 787 secondes [1].

Borne supérieure (UB) L'UB chute en trois paliers discrets : de 2 466 à 2 447 dès $t \approx 8$ s, puis à 2 442 vers $t \approx 51$ s, et enfin à 2 437 vers $t \approx 827$ s. Après ce troisième saut, aucune meilleure solution entière n'est trouvée pendant près de 960 secondes supplémentaires, ce qui confirme que l'heuristique MIP à fréquence fixe ne parvient plus à franchir ce plateau.

Borne inférieure (LB) La LB progresse de façon quasi continue mais lente : elle passe de 2 370 à la racine à 2 386 en fin d'exécution, soit un gain total de seulement 16 unités en 1 787 s. Cette progression globalement monotone est ponctuée d'oscillations locales.

On peut légitimement s'interroger sur la variation de cette borne inférieure, qui devrait en toute logique être parfaitement monotone. Ce phénomène est en réalité dû à un artefact dans notre méthode de calcul de cette borne. En effet, nous récupérons comme borne inférieure globale la valeur du nœud au sommet de notre file de priorité. Or, avec notre stratégie adaptative d'arrêt de la génération de colonnes, la borne inférieure d'un nœud tout juste évalué est souvent sa borne lagrangienne (qui peut être jusqu'à 1 % plus basse que la valeur réelle de la relaxation continue). S'il s'avère que la valeur réelle de la relaxation continue du nœud traité est identique à celle de son père (ce qui est très probable vu la faible évolution globale de la borne), l'ajout dans la file de priorité d'un fils avec une borne (lagrangienne) plus basse que celle de son père provoque cet effet d'oscillation contre-intuitif.

Gap résiduel Le gap de 2,24 % (LB = 2 386, UB = 2 437) traduit l'absence de fermeture entre les deux bornes : la LB ne monte pas assez vite pour rejoindre l'UB, et l'UB ne descend

1. Le *run* complet dure 3 607 s ; la courbe couvre la portion pour laquelle les données de traçage sont disponibles.

plus après $t \approx 827$ s. L'écart de 51 unités restant suggère soit l'existence d'une solution entière non encore découverte de valeur comprise entre 2 386 et 2 437, soit une borne inférieure encore trop lâche en raison de la fractionnalité persistante de la relaxation.

4.5 Comparaison avec l'algorithme de génération de colonnes initial

Afin d'évaluer les apports de notre nouvelle architecture et de l'intégration du *Branch-and-Price*, nous avons comparé nos résultats avec ceux obtenus par l'ancien code fourni. Ce dernier repose sur une heuristique de type *Price-and-Branch* : il effectue une génération de colonnes à la racine pour obtenir la relaxation continue, puis fige le *pool* de colonnes générées pour résoudre le problème maître restreint en variables binaires à l'aide de Gurobi.

La comparaison a été menée sur deux instances de tailles très différentes : `small_station-scenario_1` et `Variant_station-scenario_4`. Le tableau 3 résume les métriques clés.

TABLE 3 – Comparaison des performances entre l'ancien code (CG + heuristique entière) et notre *Branch-and-Price* (BnP).

Instance	Méthode	Root LB	UB finale	Gap (%)	Temps (s)	Trains pénalisés (*dummy*)
`small_1`	Ancien code (CG)	1959,5	3 669,0	46,59	$\approx 1,0$	1 (train 2)
	Notre BnP	**1959,5**	**1 965,0**	**0,00**	**0,78**	**0**
`Variant_4`	Ancien code (CG)	43 850,0	49 224,0	10,91	269,0	7 (trains 2, 3, 9, 10, 11, 15, 16)
	Notre BnP	**43 850,0**	**44 022,0**	**1,07**	3 824,1	**0**

4.5.1 Faisabilité et qualité des solutions primales

La différence la plus frappante entre les deux approches réside dans la capacité à trouver des solutions réalisables valides. L'ancien code, en se restreignant aux seules colonnes générées à la racine, ne dispose pas d'un espace de recherche suffisant pour assembler une solution entière réalisable. Par conséquent, le solveur est contraint d'utiliser des arcs *dummy* (fortement pénalisés) pour garantir la faisabilité du problème maître.

— Sur l'instance `small_1`, l'ancien algorithme sacrifie le train 2, conduisant à une borne supérieure (UB) de 3 669, très éloignée de la valeur optimale de 1 965 trouvée par notre BnP (qui ferme le gap à 0 % sans aucun arc *dummy*).

— Sur l'instance `Variant_4`, le constat est encore plus sévère : 7 trains sur les 22 sont affectés à des chemins *dummy*. La solution entière proposée par l'ancien code (49 224) n'a donc aucune réalité opérationnelle. En revanche, notre algorithme, grâce à ses heuristiques primales et au branchement itératif, parvient à trouver une solution 100 % réalisable et d'excellente qualité (UB = 44 022).

4.5.2 Efficacité calculatoire et convergence

Bien que les deux algorithmes trouvent logiquement la même borne inférieure à la racine (1 959,5 pour `small_1` et 43 850 pour `Variant_4`), l'efficacité de la résolution de la relaxation continue diffère grandement.

D'après les logs d'exécution de `Variant_4`, l'ancien code nécessite plus de 4 minutes et 89 itérations de *pricing* pour clôturer la génération de colonnes à la racine. Avec notre nouvelle architecture, non seulement la relaxation est obtenue beaucoup plus rapidement, mais le temps nécessaire pour trouver la première solution entière réalisable (TTFS) n'est que de 23,6 secondes sur cette même instance.

Notre méthode explore 81 nœuds de l'arbre de branchement et ramène le gap d'optimalité à 1,07 % dans le temps imparti. Ainsi, notre code ne se contente pas de proposer une solution mathématiquement valide : il le fait en s'appuyant sur un traitement des nœuds performant, justifiant pleinement l'effort de refactorisation mis en œuvre en amont.

4.6 Analyse globale et limites de l'approche

Bien que la section précédente se soit concentrée sur les scénarios de base (scénario 1) afin de valider notre architecture, l'exécution du solveur sur l'ensemble du jeu d'instances, dont les résultats complets (pour celles ayant réussi à faire converger la génération de colonnes, sont détaillés dans le tableau 4 en Annexe A) révèle des dynamiques plus complexes. L'analyse de ces données permet de mettre en lumière les véritables forces de notre implémentation, mais aussi d'identifier les principaux goulots d'étranglement de l'approche *Branch-and-Price* sur les instances fortement contraintes.

4.6.1 Forces de l'implémentation

Efficacité et robustesse des heuristiques primales (TTFS) L'un des succès majeurs de notre solveur réside dans sa capacité à toujours fournir une solution réalisable, quelle que soit la taille de l'instance. La colonne **TTFS** (*Time To First Solution*) illustre parfaitement cette robustesse : sur l'instance massive `large_station_sce_2` (45 trains, 116 927 arcs), le solveur trouve une première solution entière valide en 371 secondes. Sur `Variant_station_sce_4` (76 682 arcs), ce temps tombe à 23,6 secondes. D'un point de vue purement opérationnel, le solveur est capable de fournir rapidement un plan de routage exploitable, même lorsque la preuve d'optimalité est hors de portée.

Scalabilité du problème de *Pricing* Contrairement à de nombreux algorithmes de génération de colonnes où la résolution du sous-problème s'accapare la majorité du temps de calcul, notre implémentation du *pricer* par programmation dynamique s'avère extrêmement performante, même sur de très grands graphes. Par exemple, sur l'instance `medium_station_sce_4` (80 539 arcs), le temps cumulé passé dans le *pricing* n'est que de 198 secondes sur un temps total d'exécution de 5 074 secondes. Cela démontre que la construction du graphe et la recherche de plus courts chemins à coût réduit ne constituent pas la limite de notre modèle.

4.6.2 Faiblesses et goulots d'étranglement

Décorrélation entre taille brute et complexité combinatoire Les données montrent de manière contre-intuitive que le nombre de trains ou d'arcs ne définit pas la difficulté de l'instance, mais que celle-ci est dictée par sa structure (probablement la densité des conflits et les capacités des stations). En effet, l'instance `small_station_sce_2`, avec seulement 12 trains et 22 883 arcs, termine avec un gap colossal de 69,17 % après une heure de calcul et 396 nœuds explorés. À l'inverse, l'instance `Variant_station_sce_5`, pourtant bien plus imposante (26 trains, 98 052 arcs), termine avec un gap très restreint de 0,94 %. Les scénarios 2, 3 et 4 engendrent une forte fractionnalité de la relaxation continue que le branchement peine à résoudre.

L'écueil du problème maître et l'enlisement de l'arbre Le véritable goulot d'étranglement de notre solveur se situe dans la résolution du problème maître restreint (RMP) et

l'exécution des heuristiques MIP globales par Gurobi. À mesure que l'arbre s'approfondit et que le *pool* de colonnes s'enrichit, le modèle devient extrêmement lourd à résoudre. Ce phénomène provoque un effondrement du *Node Throughput* (le nombre de nœuds traités par seconde). Sur les instances `medium_station_sce_3`, `medium_station_sce_4` et `large_station_sce_2`, le solveur n'explore que **6 nœuds** en des temps d'exécution allant de 5 000 à 6 000 secondes. Traiter un seul nœud prend donc en moyenne près de 15 minutes. Dans ces conditions, l'exploration de l'arbre de recherche est virtuellement à l'arrêt.

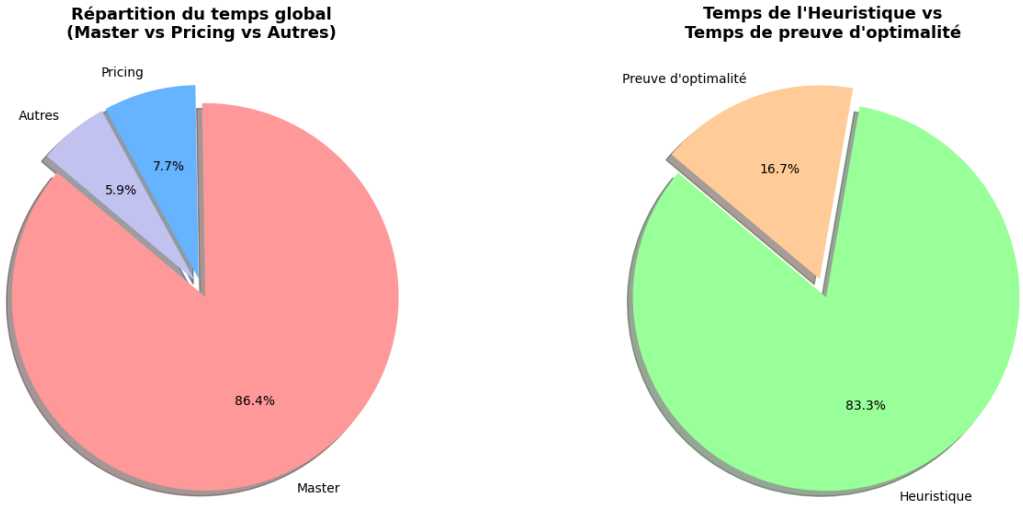

FIGURE 2 – À gauche : Répartition du temps de résolution global de la boucle de génération de colonnes. À droite : Proportion du temps alloué à la recherche de solutions primales (Heuristiques) face à la progression de la borne (Preuve d'optimalité).

Cette dynamique est visuellement confirmée par la Figure 2, qui propose une double lecture de l'allocation moyenne du temps de calcul sur l'ensemble de nos instances.

Le graphique de gauche illustre la répartition structurelle du *Branch-and-Price*. De manière frappante, le sous-problème de *Pricing*, qui contient pourtant notre algorithme de plus court chemin avec contraintes de ressources, ne représente que 7,7 % du temps total. À l'inverse, l'écrasante majorité des ressources (86,4 %) est consacré à la résolution du problème maître (RMP) par Gurobi. Les opérations d'orchestration (gestion de l'arbre, structure de données incrémentale, *Autres*) s'avèrent très économes avec seulement 5,9 %. Le nombre d'itérations du pricing étant exactement le même que le nombre du restricted master problem cela confirme que le plus gros goulot d'étranglement de notre algorithme est la résolution du RMP sans doute handicapée par le pool toujours grandissant de colonnes.

Le graphique de droite permet de mieux comprendre l'origine de la surcharge observée dans le problème maître. En effet, en isolant le temps consacré aux heuristiques primales, on constate que 83,3% du temps total de résolution est dédié à la recherche de solutions entières. Ce résultat s'explique en partie par un choix de conception : compte tenu de la très bonne qualité de la borne inférieure à la racine, comme observé sur les instances précédentes, il a été jugé pertinent de privilégier une fréquence élevée de lancement des heuristiques afin de trouver des solutions entières de haute qualité.

Par ailleurs, cette analyse doit être mise en perspective avec le fait que la majeure partie du temps de calcul est consacrée à la résolution du RMP. En combinant ces deux observations, on peut conclure que l'heuristique la plus coûteuse en ressources est l'heuristique de diving. En effet, celle-ci nécessite, pour chaque train, la résolution d'un problème de génération de

colonnes, ce qui implique autant de résolutions du RMP et donc un coût très élevé.

Stagnation de la borne inférieure (LB) On observe une stagnation relative de la borne inférieure sur les différentes instances. Par exemple, sur `medium_station_sce_3`, elle évolue seulement de 23,064,0 à 23,073,3 en 6000 s de résolution. Ce phénomène s'explique en partie par la très bonne qualité de la relaxation à la racine (quasi égale à l'optimum pour toutes les instances qui ont été fermées), ainsi que par le nombre relativement faible de nœuds explorés sur certaines instances (ici, seulement 6).

Une analyse plus fine de la fractionnalité de ces instances pourrait guider la conception de stratégie de branchement plus efficace.

4.6.3 Bilan

L'architecture de notre *Branch-and-Price* est saine et particulièrement efficace pour générer des colonnes et trouver de bonnes bornes supérieures (solutions réalisables). Cependant, pour espérer prouver l'optimalité sur les scénarios les plus complexes (2, 3 et 4), il apparaît indispensable d'alléger la charge du problème maître. Des pistes d'amélioration concrètes incluraient la mise en place d'un mécanisme de nettoyage du *pool* de colonnes (suppression des colonnes inactives de coût réduit très positif) pour accélérer Gurobi, ou encore un déclenchement adaptatif des heuristiques primales.

A Annexe : Résultats complets des expérimentations

Le tableau 4 présente l'intégralité des résultats obtenus par notre algorithme de *Branch-and-Price* sur les différentes instances de test. Les temps de résolution sont exprimés en secondes.

Table 4 – Ensemble des résultats du *Branch-and-Price* sur toutes les instances.

Instance	Trains	Arcs	Root LB	Final LB	Final UB	Gap (%)	Temps (s)	TTFS (s)	Nœuds	Master (s)	Pricing (s)	Heur. (s)
small_station_sce_1	8	14 043	1959,5	1965,0	1965,0	0,00	0,78	0,19	17	0,39	0,22	0,41
small_station_sce_2	12	22 883	5438,2	5805,9	9822,0	69,17	3612,78	7,87	396	2374,01	28,22	2366,73
small_station_sce_3	16	31 723	20534,2	20472,7	29002,0	41,66	3601,96	75,26	115	3191,05	73,23	2606,26
small_station_sce_4	18	32 470	27762,2	27693,6	36815,0	32,94	3718,13	72,78	131	3250,29	69,18	2708,55
small_station_sce_5	24	41 465	63822,9	63612,5	80492,0	26,53	3803,93	898,78	11	3591,47	180,31	2848,45
medium_station_sce_1	15	35 573	3974,0	3974,0	3974,0	0,00	5,65	1,14	13	-0,85	1,19	4,48
medium_station_sce_2	25	57 167	10694,0	10639,2	16125,0	51,56	3697,48	41,44	31	3470,27	132,33	3540,61
medium_station_sce_3	35	76 117	23064,0	23073,3	39780,0	72,41	5910,81	1020,69	6	5697,41	196,87	4864,57
medium_station_sce_4	45	80 539	81060,6	81042,4	105873,0	30,64	5074,13	2144,47	6	4858,58	198,62	2861,49
large_station_sce_1	25	76 390	7264,0	7264,0	7264,0	0,00	32,96	2,70	23	4,19	3,89	29,36
large_station_sce_2	45	116 927	20311,5	20302,1	32937,0	62,23	6027,93	371,42	6	5596,73	414,73	5620,94
Variant_station_sce_2	14	31 586	5991,3	5971,8	8752,0	46,56	3748,18	13,16	181	3116,35	83,02	3048,54
Variant_station_sce_3	18	49 520	19896,0	19747,2	21995,0	11,38	3820,20	25,80	91	3287,17	257,30	3608,61
Variant_station_sce_4	22	76 682	43850,0	43555,5	44022,0	1,07	3824,09	23,66	81	2852,05	726,99	3707,48
Variant_station_sce_5	26	98 052	59888,1	59518,0	60076,0	0,94	8830,00	165,67	21	6830,76	1937,31	8613,16